

Daily Scholar Papers Report — 2026-05-31

2026-05-31

Daily Scholar Papers Report — 2026-05-31

[Download PDF](#)

Window covered: 2026-05-30 → 2026-05-31 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

A larger-than-usual eight-candidate batch arrived in the 24-h window: four flagged-author alerts (Zhendong Su, David Lo, Michael Pradel, Shengchao Qin) and one Scholar-Recommended batch carrying four more. Two papers land as Outstanding: an ETH Zurich performance-bug-finding paper that introduces **Branch Flip Analysis** as a white-box methodology for DBMSs (21 confirmed performance issues, up to 374.9× speedup), and a CISPA + ZJU paper that reformulates Python regression test selection as **reachability on a name-element bipartite graph** (69.90% test reduction at 99.6% safe-rate, against a state-of-the-art Python RTS baseline that is safe on only 76.6% of commits). Three more papers land in Keep — a benchmark for verifier-audited *quantitative goal persistence* in long-horizon LLM agents, a proxy-layer defense that eliminates four reentrancy attack categories on 70/70 vulnerable smart contracts, and an LLM-infilling fuzzer for the Rust compiler that found 27 confirmed bugs. Two further papers receive brief summaries; one adversarial-DL-for-vulnerability-detection paper was screened out under the saturation rubric. No user-curated self-emails were forwarded today.

Outstanding: 2 · **Keep:** 3 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Finding Performance Issues in Database Systems by Exploiting Dormant Code Paths (QueryZen)	Ba, Su	arXiv 2605.22992	PDF
1.2	Names Are All You Need: Effective and Safe Regression Test Selection for Python (NameRTS)	Wang, Pradel, Liu	arXiv 2605.25356 (PACMPL submission)	PDF
2.1			arXiv 2605.23574	PDF

#	Title	Authors	Venue	Link
	Push Your Agent: Quantitative Goal Persistence (PushBench)	Cai, Zhu, Gao, Tang, Qin		
2.2	Decoupling Reentrancy Protection from Smart Contract Implementation Logic (Sentinel)	Joshi, Golab	arXiv 2605.25207	PDF
2.3	ClozeMaster: Fuzzing Rust Compiler by Harnessing LLMs for Infilling Masked Real Programs	Gao, Yang, Sun, Wu, Zhou, Xu	arXiv 2605.00413	PDF

1. Outstanding

1.1 · DYNAMIC ANALYSIS · Flip one optimization branch at a time on running DBMSs — 21 confirmed perf bugs, 26.2× mean speedup, 374.9× best case

1.1 [Finding Performance Issues in Database Systems by Exploiting Dormant Code Paths](#) — Ba, Su (ETH Zurich), arXiv 2605.22992 (May 2026)

The hardest thing about hunting DBMS performance bugs is *not having a reference*. Prior tools (APOLLO, AMOEBA, CERT, PUPPY) sidestep this by comparing across DBMS versions or across semantically equivalent queries, but each method covers only one slice of optimisation behaviour and none can target *which* optimisation actually caused the slowdown. **Branch Flip Analysis (BFA)** dispenses with the external reference and instead manufactures a local one by inverting individual IF-branches that gate optimisation decisions inside the DBMS source.

Formalisation (§2). Let $B = \{b_1, b_2, \dots, b_n\}$ denote the set of conditional branches in IF statements that enable or disable optimisations. For branch $b_j \in B$, let $b_{\{j,d\}}$ and $b_{\{j,o\}}$ denote its default and alternative paths. Given a workload W , write $P(B)$ for the DBMS performance under the default branch configuration. Construct B' such that $\forall i \neq j \ (b_i = b_{\{i,d\}}) \wedge b_j = b_{\{j,o\}}$ — every branch except b_j runs its default path. If $P(B) \ll P(B')$ the alternative is strictly faster than the default, exposing a performance issue rooted in the optimisation gated by b_j . The premise is that “the default optimisation strategy should be near-optimal; otherwise a bug exists” — a calibrated null hypothesis that prior black-box methods could not state precisely.

Why flipping branches is the right primitive. The motivating evidence is empirical: of 10 historical performance bugs reproduced from APOLLO / AMOEBA / CERT / PUPPY (Table 1 of the paper), **8 can be triggered by flipping a single IF-branch**. The remaining two are also branch-driven but require flipping multiple IFs in concert. This justifies the design choice: BFA does not require deep DBMS

expertise, doesn't depend on patched older versions, and pinpoints the exact code segment responsible for the regression — a sharp lift over previous approaches that flag a slow query without saying *why*.

Implementation — QueryZen (Algorithm 1). The instrumentation rewrites each IF as

`if (condition ^ get_env("DROP") == ID)` so a single compiled binary can be steered branch-by-branch via the `DROP` environment variable, avoiding the dominant cost of per-experiment recompilation (≈ 5 minutes per cycle). A side-channel `log(ID)` (always true, so it doesn't alter semantics) records which IFs were actually executed for the query, letting Algorithm 1 enumerate *only* live branches:

```
Input: program DBMS, instrumented program DBMS', query Q
1: cost = explain(DBMS, Q)
2: time, if_list = execute(DBMS, Q)
3: for ID in if_list do
4:   SET DROP = ID
5:   cost' = explain(DBMS', Q)
6:   if cost > cost' then
7:     time' = execute(DBMS', Q)
8:     compare(time, time')
9:   end if
10: end for
```

This avoids recompilation and skips $\sim 75.8\%$ of branch flips that the query never reaches; the instrumentation overhead is under 0.1 s per TPC-H query on PostgreSQL.

Headline numbers (§6). QueryZen was evaluated on PostgreSQL 17.0, MySQL 9.0, MariaDB 11.7, and a near-tip CockroachDB. Results across TPC-H and TPC-DS:

DBMS	Confirmed	Fixed	Analysing	Max speedup	Mean speedup
PostgreSQL	4	1	0	374.9×	n/a
MySQL	9	0	0	12.1×	n/a
CockroachDB	3	0	0	24.0×	n/a
MariaDB	2	0	2	102.2×	n/a
Total	18	1	2	374.9×	26.2× mean

The flagship issue — PostgreSQL TPC-DS 4, issue #4 — is a cost-model bug in `add_path()` where reversing a single comparison turns a 50-minute query plan into an 8-second one (50m 24s 486ms $\rightarrow$ 8s 179ms). The bug had been latent in PostgreSQL for years and is undetectable by version-comparison (APOLLO) or query-equivalence (AMOEBEA) approaches because the slow plan is the default plan in every version. Each issue affects ~ 10 other TPC-H / TPC-DS queries by $\geq 10\%$, so the user-visible impact compounds. Branch-coverage of the optimiser source increases by 8.6% with the dynamic-only filter.

False-positive profile. Two confirmed false positives. Both are *trade-off-based* — e.g. forcing CTE inline gives a 64× speedup on the reported query but degrades others, and PostgreSQL developers chose the conservative default for that reason. The paper handles this honestly: it surfaces the trade-off, doesn't claim a bug, and notes that this class of false positive is rare in the 21-issue corpus.

Why it matters. BFA is the cleanest *white-box* methodology for performance testing of complex systems software in some time. The instrumentation pattern (XOR + env-var) generalises trivially beyond DBMSs

— compiler optimisation passes, JIT heuristics, scheduler decisions, and OS kernel I/O paths all have analogous IF-gated optimisation branches that would benefit from the same single-compilation switch harness. The 21 confirmed bugs against four mature, heavily-tested DBMSs is the strongest practical validation of the methodology; the “core concept of BFA is simple and broadly applicable” framing in the abstract is rare in a performance-bug-finding paper and is, on the evidence, accurate.

1.2 · STATIC ANALYSIS · Python RTS as reachability on a name-element bipartite graph — 69.90% test-skip rate at 99.6% safe-rate, vs file-level RTS at 76.6% safe-rate

1.2 Names Are All You Need: Effective and Safe Regression Test Selection for Python — Wang, Pradel, Liu (ZJU + CISPA), arXiv 2605.25356 (May 2026)

The Python RTS landscape is awkward: dynamic typing breaks call-graph precision, and eager imports inflate file-level dependency analysis to the point that ~80% of source files appear “reachable” from every test. NameRTS sidesteps both problems by avoiding call graphs entirely and dropping the granularity below file level — without paying the call-graph precision cost.

Model (§3). A Python project is encoded as a bipartite graph $G = (E \cup N, D \cup U)$ where E are **code element nodes** (classes, functions, global variables) and N are **name nodes** (every identifier used to reference a code element). Definition edges $D \subseteq N \times E$ map names to the elements they define; usage edges $U \subseteq E \times N$ capture which names each element references. RTS becomes a reachability question: a test t is selected when any modified code element $e \in M$ lies in the set of elements reachable from the names used in t . Reachability uses a per-test working set initialised from the test’s external names + the imported modules’ `usedNames`, then propagates via definition and usage edges.

Algorithm 1 — Name-based Dependency Propagation.

```
Input: T (test files); AN (accessed names); Def (def edges); Use (usage edges);
      M (modified elements); Visible (per-test visible elements).
T' ← ∅
for t ∈ T do
  reachable_names ← external names used in t
  Modules_t ← { m ∈ Visible[t] | m.type = Module }
  for m ∈ Modules_t do
    reachable_names u= m.usedNames
  reachable_names u= AN[t]
  reachable_elements ← ∅
  work_stack ← reachable_names
  repeat
    while work_stack non-empty do
      n ← pop(work_stack)
      E ← { e | (n, e) ∈ Def ∧ e ∈ Visible[t] }
      E ← { e ∈ E | e.class = null ∨ e.class ∈ reachable_names }
      prune E with Algorithm 2
      reachable_elements u= E
      for e ∈ E do
        names ← { n' | (e, n') ∈ Use }
        add new names to reachable_names and work_stack
  refill work_stack with names of Function and SharedVariable
```

```

    elements defined in classes
until no new names added
if  $\exists e \in \text{reachable\_elements}$  s.t.  $e \in M$  then
     $T' \leftarrow T' \cup \{t\}$ 
return  $T'$ 

```

Two pruning strategies. The propagation is conservative by default — a single name with many definitions can pull in a large subgraph. NameRTS narrows this with two safety-preserving filters: (1) *critical-function pruning* — for any name corresponding to more than $N = 500$ distinct code elements, NameRTS intersects the candidate element set with the set actually *invoked* by the test in a prior execution, falling back to the unpruned set if no invocation evidence exists; (2) *context-aware element pruning* — names tagged `non-attr` are resolved against the local file first, then against explicit imports; `sure-attr` names are filtered to elements whose defining class is in the referencing class’s hierarchy; `amb-attr` names are kept unfiltered to preserve safety.

Headline numbers (§4, Table 2). Ten real-world Python projects (sympy, scikit-learn, matplotlib, dask, xarray, sphinx, pylint, seaborn, pvlb, loguru), 50 commits each, total 500 commits with ground-truth derived via instrumentation + language-server reference tracing — the first Python RTS dataset with a per-commit ground truth.

Technique	Test Reduction	Time Reduction	Safe Rate
Ground truth	80.69%	—	100.0%
NameRTS	69.90%	45.59%	99.6%
BabelRTS (file-level, prior SoTA)	28.36%	21.95%	76.6%
EkstaP (safe file-level baseline)	7.42%	3.10%	100.0%

NameRTS reaches 86.63% of the ground-truth-achievable test reduction ($69.90 / 80.69$), while maintaining safety on 498 of 500 commits. The two unsafe commits are diagnosed: one is a pylint case where a global cache in `astroid.MANAGER` masks invocation evidence (only because pytest-xdist isolation is incompatible with pylint), and one is a sphinx callback hardcoded inside the external `docutils` library that NameRTS does not analyse.

Why the prior baselines fail. EkstaP’s file-level analysis ekes out 7.42% reduction because Python’s eager imports mean every test transitively imports ~83.27% of source files. BabelRTS gets higher reduction (28.36%) by ignoring implicit parent-package and `importlib`-dynamic imports — but that’s exactly the source of its safety drop to 76.6%. NameRTS’s bipartite-graph reachability is precise enough to skip irrelevant tests *and* general enough to track the dependencies BabelRTS misses. The matplotlib row is the cleanest case: BabelRTS hits 97.63% test reduction at 14.0% safe rate (i.e. almost always wrong); NameRTS hits 54.43% at 100.0% safe rate.

Overhead (Table 3). Total NameRTS overhead averages 10.04% of full-suite runtime (4.45% select + 3.22% init + 2.38% runtime monitoring), against 44.37% saved on test execution. Net wins of ~34% per-commit even when accounting for analysis cost.

Why it matters. Function-level / call-graph RTS has long been the “right answer” for statically typed languages (Ekstazi for Java, Hyrise-RTS for C++) but the literature has never had a working analogue for Python. NameRTS shows that the bipartite-graph reformulation — bypassing call graphs entirely — gives the precision-by-construction of function-level RTS *without* requiring sound call-graph inference, which is

open-problem-hard for Python. The transferable lesson is structural: when call graphs are too unsound to use, lifting the analysis to *names rather than functions* recovers most of the precision because the language’s binding semantics already does the function-resolution work at runtime. Direct ports to TypeScript-with- `any` , Ruby, and JavaScript with dynamic `require` look immediately viable.

2. Keep

2.1 · LLM AGENTS · QGP benchmark separates verifier-audited progress from final-success — STATEQGP controller lifts agent persistence from $\leq 30\%$ to 69–78% at zero duplicate-submit rate

2.1 [Push Your Agent: Measuring and Enforcing Quantitative Goal Persistence in Long-Horizon LLM Agents](#) — Cai, Zhu, Gao, Tang, Qin (Independent + Xidian), arXiv 2605.23574 (May 2026)

The framing question is sharp: long-horizon LLM agents routinely *appear* to be doing useful work, yet stop short of the requested count. Existing benchmarks score pass/fail or recall@N and so collapse three distinct failure modes — premature stop, duplicate submission, false completion — into a single binary outcome. **PushBench** decomposes them by routing every “work unit” through an external verifier and tracking the verifier-confirmed count.

Definitions (§3). Let D_t be the set of items the agent has submitted by step t , and V an external verifier. Then:

$$\text{valid_count_t} = |\{ x \in D_t : V(x) = 1 \}| \quad (1)$$

Success requires $\text{valid_count_t} \geq \text{target_count} = N \quad (2)$

The agent additionally reports a self-believed count reported_count_t . The benchmark measures four failure modes:

- *False completion:* final action with completion claim while $\text{valid_count_t} < N$.
- *Premature stopping:* ask-user or non-completing final action with $\text{valid_count_t} < N$.
- *Reported-count error:* $|\text{reported_count} - \text{valid_count}| / \text{target_count} \quad (3)$
- *Duplicate work:* duplicate submissions visible to the verifier.

Two controllers. *STATEQGP* (state-tracking, for retrieval tasks) maintains externally visible state over submitted IDs, search pagination, and completion eligibility. *UNITQGP* (backlog-tracking, for work-unit tasks) keeps verifier-visible state over submitted units, accepted/rejected outcomes, remaining target count, and termination eligibility. Both refuse to terminate while $\text{valid_count} < N$; verifier gating alone (without the duplicate-aware progress state) is shown to be insufficient.

Headline numbers (Table 1, QGP-RepoScan). Three backends (gpt-4.1-mini, gpt-4.1, gpt-5.4), two agent shells (Native, LangGraph), four controllers (Standard, Verifier-gated, STATEQGP, LETTA/MEMGPT memory baseline, LG+Memory), evaluated on 36 fixed task instances per row at target counts $N \in \{10, 25, 50, 100\}$:

Model	Standard	Verifier-gated	STATEQGP	Memory (LG+Memory)
gpt-4.1-mini Native	2.8%	8.3%	72.2%	5.6%
gpt-4.1 Native	30.6%	33.3%	72.2%	47.2%

Model	Standard	Verifier-gated	STATEQGP	Memory (LG+Memory)
gpt-5.4 Native	30.6%	47.2%	69.4%	72.2%
gpt-5.4 LangGraph	16.7%	22.2%	77.8%	72.2%

Duplicate submit rate under STATEQGP is **0.000 across every model-backend pair**, against 0.557–0.881 under Standard. Memory baselines (LETTA/MEMGPT, LangGraph+Memory) catch up at gpt-5.4 but lag substantially on smaller models — memory is helpful but model-dependent and not a uniform substitute for verifier-aligned progress state.

Work-unit results (Experiment 2, QGP-DataOps-lite). When each item must be inspected, checked / edited / answered, verified, then submitted, the contrast sharpens: Standard and Verifier-gated controllers complete **zero** task instances across all three models, while UNITQGP restores 50.0% / 29.2% / 25.0% success for gpt-4.1-mini / gpt-4.1 / gpt-5.4. The interpretation is that local task competence and quantitative-goal reliability stress different mechanisms; agents need explicit progress-state tracking to handle the latter.

Frontier-agent black-box check. Claude Code (Sonnet 4.6) and Codex CLI (gpt-5.4) solve many 50-artifact tasks but drop to **3/9 successes per condition at 100 artifacts**, confirming that even the strongest current agent shells exhibit the QGP failure mode at moderately larger targets.

Why it matters. The benchmark is the contribution: by making the verifier’s count the only authoritative signal and making “stop only when the requested work is complete” an explicit task contract, PushBench surfaces failure modes that the SWE-bench / AgentBench style of evaluation hides. The STATEQGP / UNITQGP controllers are sketched as one *implementation* of the principle (controller-level externally-visible progress state) — others are presumably possible, but the matched-controller comparisons make the case that this is the cheapest way to lift agent reliability on quantitative goals. Directly relevant to any production agent that ships “collect N things” tasks: alert / report aggregation, multi-document retrieval, batch refactoring.

2.2 · SMART CONTRACTS · Proxy-layer reentrancy guard with dual-mode (per-contract + cross-contract registry) — 70/70 coverage across four reentrancy attack categories, ~25k gas overhead

2.2 Decoupling Reentrancy Protection from Smart Contract Implementation Logic — Joshi, Golab (Waterloo), arXiv 2605.25207 (May 2026)

The conventional Solidity reentrancy guard (OpenZeppelin’s `ReentrancyGuard`) is a single-contract mutex, which covers Single-Function Reentrancy (SFR) but leaves Cross-Function Reentrancy (CFR), Cross-Contract Reentrancy (CCR) and Read-Only Reentrancy (ROR) under-protected. **Sentinel** moves the guard out of the implementation contract entirely and into a proxy layer that intercepts every external call.

Formalisation (§III). A contract is modelled as $C = (S, F, T)$ with state S , functions F , and transitions T . The paper gives four definitions:

- **Definition III.1 — Single Function Reentrancy (SFR).** A function $f \in F$ is reentered before its first invocation completes, leaving S in an inconsistent intermediate state.
- **Definition III.2 — Cross Function Reentrancy (CFR).** During execution of $f_1 \in F$, an external call enables an attacker to invoke $f_2 \in F$ that operates on a shared state variable not yet updated by f_1 .

- **Definition III.3 — Cross Contract Reentrancy (CCR).** Reentrancy along $f \rightarrow \text{ExtCall}(C', f1')$ where C' is a cooperating attacker contract whose execution path reenters back into C .
- **Definition III.4 — Read Only Reentrancy (ROR).** A staticcall to a view function returns stale state because an in-progress state transition has not yet committed.

Architecture. SentinelProxy intercepts every call to the implementation. Algorithm 2 is the main execution flow: detect staticcall via Algorithm 3 (which uses a log0 emit-and-catch trick that reverts inside staticcall context, letting the proxy distinguish call vs staticcall without storage writes), then route to either `_optimizedGuard()` (Algorithm 4, single-contract mutex) or `_highSecurityGuard()` (Algorithm 5, external ILockRegistry that coordinates locks across multiple proxies). The optimized mode covers SFR / CFR; the high-security mode adds CCR coverage via the shared registry. Both modes lock on staticcall paths to prevent ROR.

Headline numbers (SV, dataset of 70 vulnerable contracts).

Mitigation	SFR	CFR	CCR	Total
Sentinel	38/38	20/20	12/12	70/70
OZ ReentrancyGuard	38/38	13/20	0/12	51/70
LiqGuard	38/38	7/20	0/12	45/70

Sentinel achieves complete coverage across all four attack categories, including the 12/12 CCR cases that OpenZeppelin's per-contract mutex cannot reach. The detection-tool comparison column (Slither 71.42%, Mythril 64.28%, Oyente 57.14% true-positive rate on the same corpus) is fairly the wrong axis — those tools identify rather than prevent — but it underscores the gap between detection and deployable defense.

Gas profile. Staticcall overhead 6,400–7,500 gas; optimized mode adds ~25,000 gas; high-security mode (external registry) adds ~53,000 gas. Set against a typical DeFi transaction at 200,000–500,000 gas, optimized mode is 5–12.5% overhead and high-security mode 10–25% — a modest cost for the broader coverage.

Why it matters. Smart-contract reentrancy detection has been a saturated topic for years; *deployable* reentrancy *defense* that actually closes CCR has been the gap. Sentinel's contribution is architectural: by decoupling the guard from implementation logic and standing up a cross-contract registry, it makes the cross-contract case tractable without rewriting deployed contracts (because proxies are upgrade-points by design). The Algorithm 3 staticcall detection trick — `log0` inside a try/catch — is the most transferable component for any Solidity infrastructure that needs to distinguish call from staticcall without paying for storage. Read alongside today's adversarial-DL reentrancy paper, the contrast is sharp: the detection side is at saturation; the defense side still has structural moves left.

2.3 · FUZZING · LLM-infill at bracket positions of historical bug-triggering Rust snippets — 27 confirmed rustc/mrustc bugs, 10 fixed, higher coverage than baseline LLM fuzzers

[2.3 ClozeMaster: Fuzzing Rust Compiler by Harnessing LLMs for Infilling Masked Real Programs](#) — Gao, Yang, Sun, Wu, Zhou, Xu, arXiv 2605.00413 (May 2026)

Directly asking an LLM to emit Rust often produces syntactically invalid programs — Rust's lifetime / borrow grammar is unfriendly to free-form generation. ClozeMaster instead lifts the structure of test cases that *historically triggered* rustc / mrustc bugs and infills only the bracket-enclosed regions.

Motivating analysis. Of 5,242 closed bug-labeled rustc issues from January 2019 to August 2023, the average issue takes ~209 days to close. A bracket-coverage analysis over the bug-triggering Rust snippets shows pervasive use of all four bracket families: `()`, `{}`, `[]`, and `<>`. The mask-then-fill design is motivated by this observation: bracket positions are where Rust's interesting structural variation lives.

clozeMask (Algorithm 1). Two phases. *Cloze*: a stack-based DFS locates every matched bracket pair in a seed snippet and produces a family of masked variants, each with one bracket region replaced by `__mask__`. *Infilling*: an InCoder-style LLM (fine-tuned on the 100,000 augmented Rust snippets dataset via Adam, ~48h training) fills each masked position. Special bracket positions inside `#[feature(...)]` blocks receive multiple fill attempts at varied temperatures to broaden coverage. Outputs that are identical to the original snippet are discarded.

Two oracles. *ICE* — Internal Compiler Error — is detected when rustc emits “internal compiler error” or “compiler unexpectedly panicked” (or mrustc emits “BUG” + core dump). *Hang* — infinite loop / unbounded resource consumption — is detected by stack-trace + duration thresholds. Duplicates are filtered via stack-trace + time-passes clustering.

Headline numbers. 37 reported bugs, 27 confirmed by developers, **10 fixed**. ClozeMaster outperforms existing fuzzers on code coverage and on confirmed-bug count on both rustc and mrustc.

Why it matters. The “infill historical bug-triggering programs” recipe is a different style of LLM-fuzzing from the dominant “ask the LLM to generate from scratch” school (Titanfuzz, FuzzGPT). It pays a smaller LLM bill (only the masked positions need generation), inherits validity from the surrounding context, and concentrates probing pressure on the structural positions most associated with historical bugs. The bracket-pair DFS + per-position infill is a clean primitive that should transfer to other strict-grammar compilers (Haskell, OCaml, Solidity).

3. Brief Summaries

3.1 · AGENT SECURITY · Position-paper synthesis on prompt injection turning agentic coding assistants into the attacker's shell

3.1 [How Agentic AI Coding Assistants Become the Attacker's Shell](#) — Liu, Zhao, Lyu, Zhang, Wang, Lo (SMU + others), arXiv 2605.25871 (May 2026)

A short synthesis / position piece on how unvetted external artifacts (READMEs, issue bodies, third-party docs, web search results) carry hidden instructions that hijack agentic coding assistants into running unauthorised commands. The paper “examines how these prompt injection attacks work, measures their prevalence, discusses the limitations and challenges of current defenses, and suggests future research directions”. Useful framing of the agent-side attack surface; the operational measurement work is in companion threads, not this article. Treated as awareness reading.

3.2 · LLM EVALUATION · Prompt-strategy sensitivity audit for LLM vulnerability detection — chain-of-thought wins, self-consistency over-abstains

3.2 [PromptAudit: Auditing Prompt Sensitivity in LLM-Based Vulnerability Detection](#) — Camarato, Hmaiti, Ghadamian, Mohaisen — arXiv 2605.24171 (May 2026)

A controlled-evaluation paper that fixes dataset, decoding, and parsing while varying only the prompting strategy across five open-weight models on 1,000 CVEs (6,074 code samples across 16 languages). The

five strategies are standard chain-of-thought, few-shot, adaptive chain-of-thought, self-consistency, and a baseline. Headline findings: standard CoT achieves the strongest operational performance, few-shot helps for prompt-sensitive models, adaptive CoT suppresses recall, and self-consistency drives excessive abstention that crushes effective F1. The artifact value is the framework itself — a way to isolate prompt effects from model and dataset effects in vulnerability-detection benchmarks — rather than any specific deployment recommendation. Methodologically reusable; substantive novelty modest.

Cross-Paper Synthesis

The two Outstanding papers operate at opposite ends of the analysis-target spectrum — runtime DBMS performance vs static Python dependency analysis — but share a pattern: *replace a hard-to-construct ground truth with a precise, locally-constructed reference*. QueryZen manufactures the reference by branch-flipping the DBMS itself; NameRTS manufactures the reference by reformulating reachability over names rather than functions. Both succeed because the conventional ground truth (cross-version perf comparison, sound call graphs) is either unavailable or too expensive, and both authors found a representation in which the precision-vs-safety trade-off lands far better than in the conventional formulation. This is the strongest variant of “right model of the problem beats more compute on the wrong model” — the BFA primitive needs no DL, and the bipartite name-element graph needs no symbolic call-graph solver.

The two Keep agent/security papers — PushBench and Sentinel — share an *architectural* take on reliability that contrasts with the dominant *model-centric* line. PushBench argues that LLM agent reliability on quantitative goals is best achieved by an external controller that owns the progress state rather than by training a better policy. Sentinel argues that smart-contract reentrancy is best closed by routing every call through a separate proxy layer rather than by improving the per-contract guard. Both prefer “fix it in the wrapper” over “fix it in the core”. The transferable thesis is that both LLM agents and Solidity contracts have weak introspection / state-tracking primitives, so externalising the cross-cutting concern (progress tracking; cross-contract locking) is more robust than retrofitting it inside.

ClozeMaster (Keep 2.3) is methodologically aligned with the same “structure-from-history” line: rather than asking the LLM to invent Rust programs, it asks the LLM only to vary positions inside snippets that historically triggered bugs. The Titanfuzz / FuzzGPT axis (generate-from-scratch) and the ClozeMaster axis (infill-from-history) are complementary, and a hybrid where the LLM infills positions inside a coverage-guided mutator’s selected seeds is the obvious next step.

The recommended-bucket adversarial-DL reentrancy paper continues the saturated DL-for-vuln-detection trend; the contrast with Sentinel’s proxy-layer defense in the same batch makes the synthesis sharper than usual: the detection side has plateaued, the defense side still has architectural moves.

Writing & Rationale Insights

QueryZen demonstrates the value of a **calibrated null hypothesis** as a paper’s structural anchor: “the default optimisation should be near-optimal” is precise enough to be falsifiable on every individual branch, simple enough to fit on a slide, and rare enough in the performance-bug literature to land as a methodological contribution rather than an engineering ablation. Authors writing performance / correctness testing papers should ask, “What is the precise null I am testing, and can I make it locally

constructible?” — and if the answer reaches outside the system being tested (cross-version, cross-tool), the paper will be classified as engineering. QueryZen’s null is *inside* the DBMS, and that is the whole methodological lift.

NameRTS illustrates a different writing pattern: **define the right structural representation, then let the algorithm fall out of it**. The bipartite name-element graph is doing the heavy lifting in §3; Algorithm 1 is essentially a textbook BFS over that graph once the model is in place. The paper spends its pages building intuition for the graph (Figure 1, the worked example, the pruning sections) rather than over-describing the algorithm. This is the right ratio when the contribution is a representation rather than a procedure.

PushBench shows a useful **decomposed-failure-modes framing**: rather than reporting a single “task success” rate, the paper enumerates four distinct failure modes (false completion, premature stopping, reported-count error, duplicate work) and reports them per row. This makes the matched-controller comparison crisp — STATEQGP doesn’t just lift success rate, it drives the duplicate-submit rate to zero, which is the cleaner-than-success qualitative shift. Benchmark papers should consider whether their headline metric is hiding a more informative decomposition.

Sentinel’s strongest writing decision is the **four definitions on a single page** (Definitions III.1–III.4): it lets the reader hold the full taxonomy of reentrancy attacks in one place before the algorithm-by-algorithm walk-through. Defense papers that lack this kind of taxonomy-first framing risk being mistaken for one more reentrancy-guard variant rather than a contribution to the taxonomy itself.